gomesf@chalmers.se
Computer Science and Engineering Department
Chalmers and the University of Gothenburg

DAT321 DIT843
Mutation Testing

Dr. Francisco Gomes de Oliveira Neto (he / him)

Alan Turing

Father of theoretical computer science and artificial intelligence.

Officer of the Order of the British Empire. Wartime Code Breaker.

LGBTQIA+ Icon!

What we will learn today?

- What is **Mutation Testing**?
- How can we **use mutants** to check the quality of the test suite?
- What different **mutation operators** can we use?
- What are **equivalent mutants**? How do they affect the **mutation score**?
- What are the trade-off of **mutation testing tools**?

© Francisco Oliveira Neto - 2018

Part 1:

Mutation Testing!

What, How and Why?

Mutation Testing - Overview

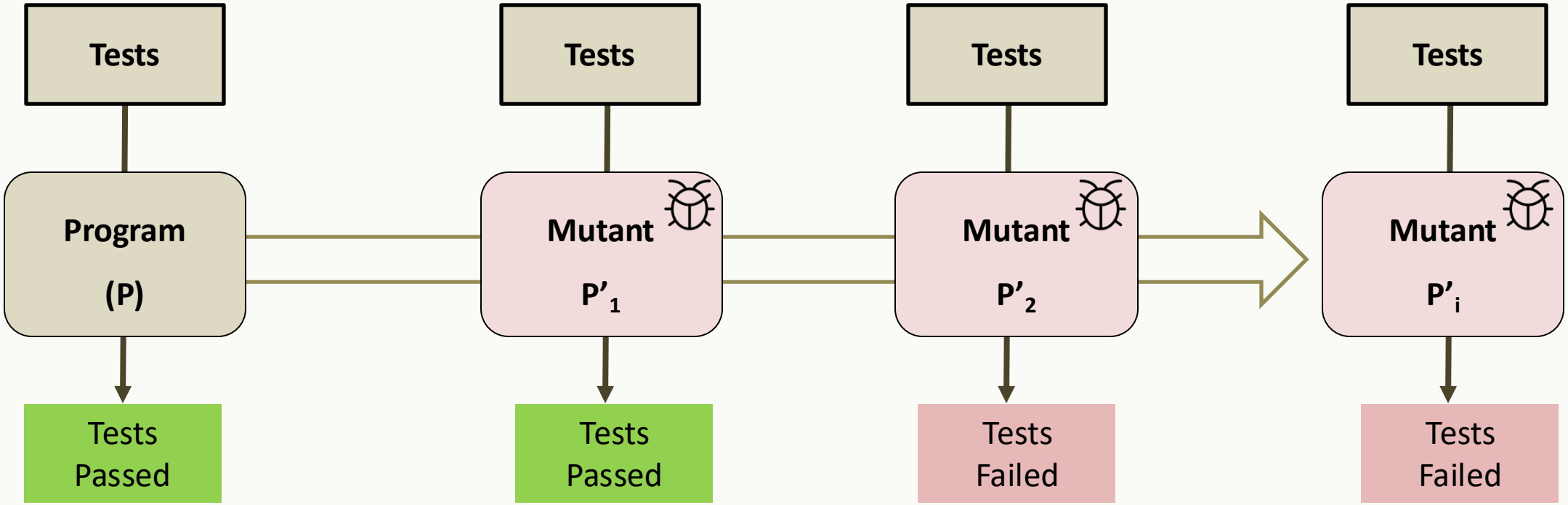
- **Main purpose:** The goal is to check the **quality of the tests**.
- **Core idea:** We create **faulty versions** of the software and re-run the tests on those faulty versions. We **expect the tests to fail**, since we know that there are faults in the code. Tests that pass could not detect the fault, hence indicating that they may not be good tests.

- **Mutant**: Given a program *P*, a mutant *P'* is obtained by introducing a *syntactic change* to P.
- **Syntactic Change**: A small *change* in the code that should make the code still valid (runs).
- **Change**: A change to the code that mimics typical human mistakes.

© Francisco Oliveira Neto - 2018

Mutation Testing - Overview

In the example below, mutant **P1 survived**, but mutant **P2 and Pi were killed** by the tests.



© Francisco Oliveira Neto - 2018

Mutation Testing – Mutation Score:

- $\mathbf{MC:\ MutationCoverage} = \frac{\mathbf{mutants\ killed}}{\mathbf{total\ mutants}}$

- Ideally, a good test suite kills all mutants in the code **(100% MC)**.
- **Note:** Mutation coverage does not tell us how good the code, only how good the test suite is. MC is a Test Adequacy criteria.

- Therefore, **mutation coverage is a complement to code coverage.**

© Francisco Oliveira Neto - 2018

Mutation operators

- Mutation operators are grammatical **rule** to introduce a **syntactic change**.

- **Real fault based operators:** Creates faults that look like common mistakes made by programmers. Should be independent of language.
- **Language-specific operators**: Creates faults specific for a certain programming language. For example, changes related to the inheritance feature we have in Java, or changes regarding pointer manipulations in C.

© Francisco Oliveira Neto - 2018

Mutation Operators - Procedural

ID	Mutation Operator	Description	Examples
AOR	Arithmetic Operator Replacement	Replaces an **arithmetic operator** by another.	`+, -, *, /, %.`
ROR	Relational Operator Replacement	Replaces a **relational operator** by another.	`<=, >=, !=, ==, >, <.`
COR	Conditional Operator Replacement	Replaces a **conditional operator** by another	`&&, ||, &, |, !, ^`
AOR	Assignment Operator Replacement	Replaces an **assignment operator** by another	`=, +=, -=, /=`
SVR	Scalar Variable Replacement	Replaces **each variable** reference **by another variable** reference. (data flow)	**Orig:** `return x < y ? x : y` **Muta:** `return y < y ? y : y`

© Francisco Oliveira Neto - 2018

Mutation Operators - OOP - Encapsulation

- AMC – Access modifier change: Changes the **access level** for instance variables and methods. Example

```
public class ShoppingCart {
    // ...
    private List<String> products;
}
```

```
public class ShoppingCart {
    // ...
    public List<String> products;
}
```

```
public class ShoppingCart {
    // ...
    protected List<String> products;
}
```


© Francisco Oliveira Neto - 2018

Mutation Operators – OOP - Inheritance

- IHI / D – Hiding variable insertion / Deletion: Adds / Removes **hiding variables** with the same name in **subclasses**.

```
public class ShoppingCart {
    // ...
    int maxProducts;
    int minProducts;
}

public class SortedShoppingCart extends ShoppingCart{
    // ...
    int maxProducts;

}
```

deletion

```
public class SortedShoppingCart extends ShoppingCart{
    // ...
}
```

insertion

```
public class SortedShoppingCart extends ShoppingCart{
    // ...
    int maxProducts;
    int minProducts;
}
```

Equivalent and Subsumed Mutants

- **Some** mutants **can never be killed** because they trigger the same behaviour as the correct behaviour of the SUT. Those mutants are named **equivalent mutants**.

- Most tools calculate the simple mutation score (left) because we **cannot detect equivalent mutants automatically, this is an undecidable problem.**

© Francisco Oliveira Neto - 2018

Part 2:

Limitations and Benefits of Mutation Testing

Equivalent and Subsumed Mutants

- Some techniques creates combinations of mutants named **High Order Mutations** (HOM).

- One of the challenges with HOM is that some combinations will **subsume others**. Subsumed HOM mutants are those that would be killed by another more powerful combination of mutants.

© Francisco Oliveira Neto - 2018

Mutation Score and Equivalent Mutants

- We do mutation analysis by calculating the **mutation score**:
  - Simple version: $mutationScore = \frac{mutantsKilled}{totalMutants}$;
  - Avoids equivalent mutants: $mutationScore = \frac{mutantsKilled}{nonequivalMutants}$
- **Low mutation** score means that few mutants were killed, hence the **test suite must be improved** by creating new cases or revising existing ones.

Mutation Testing - Tools

- µJava: http://cs.gmu.edu/offutt/mujava/. (Java): µJava was one of the **first tools** and was very popular. Today it is not used except for research projects.

- PIT: https://pitest.org/ (Java): PIT is **fast, easy to use** with different tools (maven, graddle) and a variety of configurations (mutation operators).

- MutPy: https://pypi.org/project/MutPy/ (Python)

© Francisco Oliveira Neto - 2018

MT - Benefits and Challenges

- Empirical studies reveal that that mutation coverage **reveals additional code weaknesses** when compared to branch coverage [Parsai and Demeyer, 2020], hence being complementary to typical structural coverage.

- MT is **extremely expensive**. Each mutation on the source code level requires re-compiling and re-runing all tests. Some tools like PIT try to improve efficiency by mutating bytecode, or using heuristics to choose subsets of tests to run.

- MT is **difficult to scale**. Most research focuses on unit level, since mutations involve source code, it is hard to do them at higher levels of testing or for manual testing.

© Francisco Oliveira Neto - 2018

Paper: https://ieeexplore.ieee.org/abstract/document/9402251

Mutation Testing of Machine Learning Software

- Some papers have proposed mutation operators for ML software but the **research community is still learning** what those operators do.
- In their paper, Panichella and Liem discuss how we can still **not understand what a mutation operators for a ML** software would look like

In ML, we still lack a clear sense of what makes for simple or complex mutants. As noted in [18], the stochastic nature of training may lead to multiple re-trainings of the same model on the same data leading to models that may differ in weights and output. Also, when considering mutations after a model has been trained, not all neurons and layers have equal roles, so the effect of mutating a single neuron may greatly differ.

What Are We Really Testing in Mutation Testing for Machine Learning? A Critical Reflection

Annibale Panichella
a.panichella@tudelft.nl
Delft University of Technology
Delft, The Netherlands

Cynthia C. S. Liem
c.c.s.liem@tudelft.nl
Delft University of Technology
Delft, The Netherlands

***Abstract*—Mutation testing is a well-established technique for assessing a test suite's quality by injecting artificial faults into production code. In recent years, mutation testing has been extended to machine learning (ML) systems, and deep learning (DL) in particular; researchers have proposed approaches, tools

niques, as extensively surveyed by Ben Braiek & Khom [5], as well as Zhang et al. [6]. While this is a useful development, in this work, we argue that *current testing for ML approaches are not sufficiently explicit about what is being tested exactly.*

Exercise – Mutation in Python

- Download the zip file in Canvas: lecture_mutation_exercise.zip
- Introduce 10 mutants, from different operators.
- Calculate the mutation score for the code.

Bonus (Coverage comparison):

- Setup the Python project On the project root, run the commands:
  - Activate the environment: *source .venv/bin/activate*
  - Run the pytest-cov report: *coverage report -m pytest tests/test_score_analyzer.py*
  - What is the code coverage?

© Francisco Oliveira Neto - 2018

```python
"""
This function provides functionality to analyze a list of scores based on given threshold and bonus values.
"""
def analyze_scores(scores, threshold, bonus):

    if threshold < 0 or bonus < 0 or len(scores) < 1:
        raise ValueError("Threshold and bonus must be non-negative. Scores cannot be empty.")

    for score in scores:
        if score < 50:
            bonus = bonus - 1 # Penalty for negative scores

        elif score >= threshold:
            bonus += 1 # Reward for scores above threshold

    # Double the bonus if there are more than 3 scores and bonus is greater than 5.
    if len(scores) > 3 and bonus > 5:
        bonus = bonus * 2

    total = sum(scores) + bonus

    return total
```

```python
# Note: The following test cases were automatically generated using
#   ChatGPT based on the provided code snippet.
def test_case_1():
    assert analyze_scores([60], 50, 3) == 60 + 4

def test_case_2():
    assert analyze_scores([10, 20], 40, 2) == sum([10, 20]) + 0

def test_case_3():
    assert analyze_scores([50], 50, 1) == 50 + 2

def test_case_4():
    assert analyze_scores([30, 70], 60, 5) == sum([30, 70]) + 5

def test_case_5():
    assert analyze_scores([40, 55, 65], 50, 3) == sum([40, 55, 65]) + 4

def test_case_6():
    assert analyze_scores([25, 48], 80, 4) == sum([25, 48]) + 2

def test_case_7():
    assert analyze_scores([10, 70], 50, 2) == sum([10, 70]) + 2

def test_case_8():
    assert analyze_scores([60, 60, 60, 60], 50, 6) == sum([60, 60, 60, 60]) + 20

def test_case_9():
    assert analyze_scores([40, 55, 70], 50, 3) == sum([40, 55, 70]) + 4

def test_case_10():
    with pytest.raises(ValueError):
        analyze_scores([], 5, 2)
```